

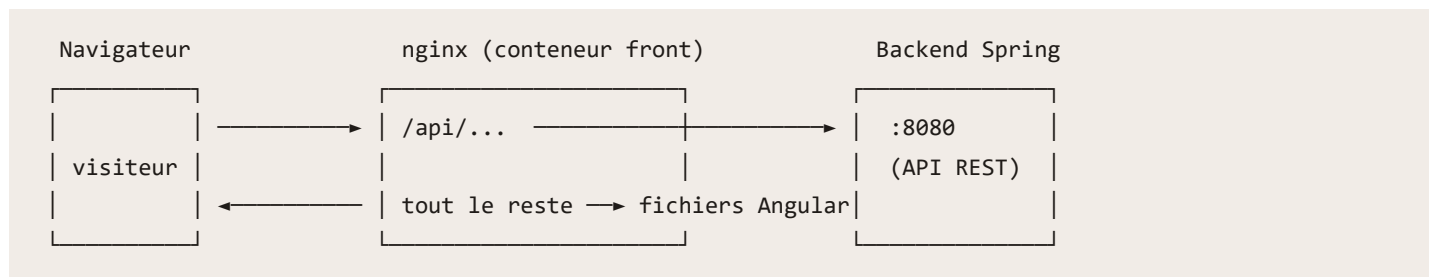
nginx reverse proxy : le bloc `location ^~ /api/`

Le navigateur ne parle qu'à un seul serveur : le conteneur nginx du front. Ce nginx sert les pages Angular lui-même, et **relaie** tout ce qui commence par `/api/` vers le backend Spring. Cette fiche décortique ce bloc ligne par ligne, avec ce qu'il fait vraiment dans DataShare et ce qu'il ne fait pas (encore).

💡 Pour un enfant de 5 ans

Tu téléphones chez mamie. C'est une **standardiste** qui répond. Si tu veux « un dessin », elle te le donne tout de suite. Si tu veux « un calcul compliqué », elle appelle un grand en cachette, écoute sa réponse et te la répète. Toi, tu n'as parlé qu'à elle.

Un nginx placé devant un autre serveur pour relayer les demandes s'appelle un **reverse proxy** (« proxy inverse » : il représente le serveur auprès des clients, à l'inverse d'un proxy classique qui représente les clients).



Le bloc complet

Source : [frontend/nginx.conf:19-26](#) .

```
location ^~ /api/ {
    proxy_pass http://backend:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_request_buffering off;
}
```

`location ^~ /api/` : l'aiguillage

`location` choisit les règles à appliquer selon le début de l'URL. `^~` signifie : « si ce préfixe correspond, **ne teste pas les règles à expression régulière** ».

Ce n'est pas théorique : le même fichier contient une règle regex pour le cache des assets, [nginx.conf:36](#) :

```
location ~* \.(?:js|css)$ { ... Cache-Control: public, max-age=31536000, immutable ... }
```

Requête : GET /api/export/stats.js

SANS ^~ (préfixe simple)

- 1. préfixe /api/ trouvé
- 2. nginx teste aussi les regex → \.js\$ correspond
- 3. la regex GAGNE X
→ cherche un fichier .js, le backend n'est jamais appelé

AVEC ^~

- 1. préfixe /api/ trouvé
- 2. ^~ : on s'arrête là
- 3. le bloc /api/ GAGNE ✓
→ la requête part au backend

🧠 Pour un enfant de 5 ans

^~ veut dire : « si ça commence par /api/, c'est pour le grand, **point final**. Ne va pas chercher une autre règle ».

proxy_pass : où envoyer, et avec quel chemin

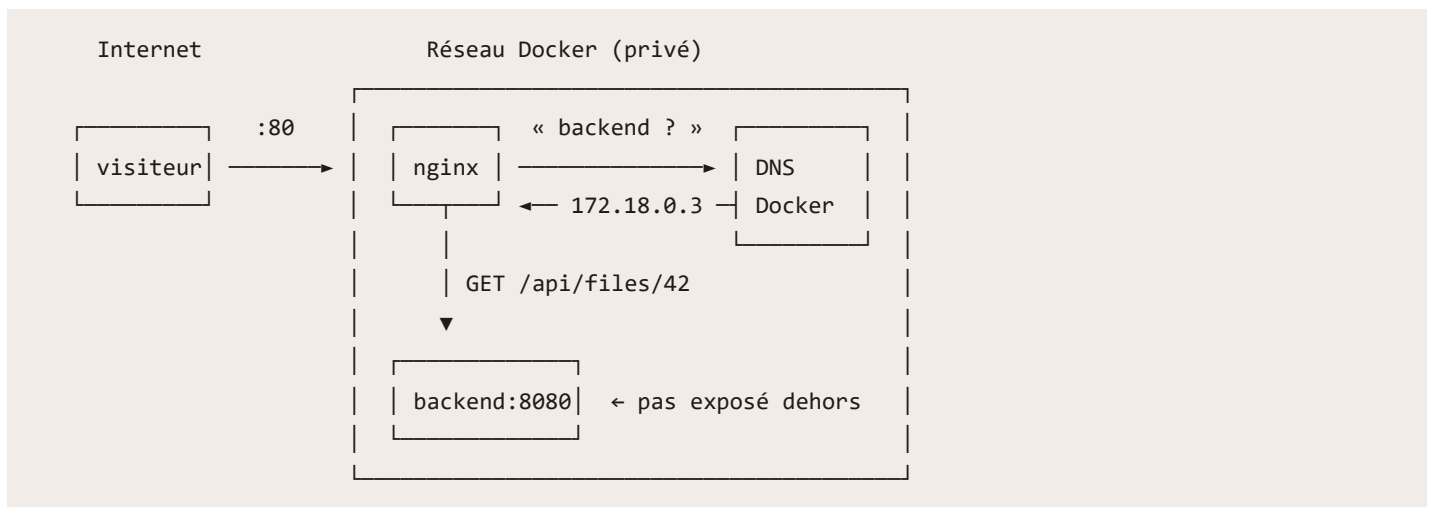
proxy_pass http://backend:8080; envoie la requête à l'hôte backend , port 8080.

Règle du chemin : sans chemin après le port, nginx transmet l'URL **telle quelle**. Avec un / final, il retirerait le préfixe.

Écriture	Reçu par nginx	Reçu par le backend
proxy_pass http://backend:8080; (DataShare)	/api/files/42	/api/files/42
proxy_pass http://backend:8080/;	/api/files/42	/files/42

C'est ce qui permet à Spring de déclarer ses routes en /api/... sans réécriture.

Le nom backend n'est pas une adresse Internet : c'est un nom de service que le DNS (Domain Name System) interne de Docker traduit en adresse IP sur le réseau du conteneur.



Conséquence sécurité : seul nginx est exposé ; le backend n'est joignable que via lui.

⚠ **À ne pas affirmer à l'oral sans nuance** : le dépôt ne contient pas d'orchestration qui déclare un service nommé `backend`. `deploy/docker-compose.yml` ne lance que Postgres, MinIO et Adminer ; `docker-compose.prod.yml` est **hors périmètre MVP** ([docs/BACKLOG.md](#) , section « Conteneurisation complète »). Le nom `backend` est donc une convention attendue par `nginx.conf` , que la future orchestration devra respecter. En développement, c'est `ng serve` qui proxifie, via [frontend/proxy.conf.json](#) (`/api` → `http://localhost:8080`).

💡 Pour un enfant de 5 ans

« `backend` » n'est pas une vraie adresse, c'est un **surnom** : dans la grande maison Docker, chacun a un surnom et un petit annuaire dit où il habite. La standardiste dit « passe-moi *le grand* », et l'annuaire trouve la bonne chambre.

Les `proxy_set_header` : les étiquettes sur l'enveloppe

Quand nginx relaie, c'est **lui** qui ouvre une nouvelle connexion vers le backend. Sans précaution, le backend croit que tout vient de nginx. Les en-têtes ajoutent l'information d'origine.

En-tête	Valeur	Exemple	Ce que ça transmet
<code>Host</code>	<code>\$host</code>	<code>datashare.exemple.fr</code>	le nom de site demandé par le visiteur
<code>X-Real-IP</code>	<code>\$remote_addr</code>	<code>82.12.34.56</code>	l'IP (Internet Protocol) du visiteur
<code>X-Forwarded-For</code>	<code>\$proxy_add_x_forwarded_for</code>	<code>82.12.34.56, 10.0.0.1</code>	la chaîne des IP traversées (chaque proxy s'ajoute)
<code>X-Forwarded-Proto</code>	<code>\$scheme</code>	<code>https</code>	le protocole utilisé côté visiteur

Visiteur 82.12.34.56 — HTTPS → nginx — HTTP → backend

Ce que voit le backend SANS étiquettes

```
client : 172.18.0.2 (nginx)
protocole : http
host : backend:8080
```

Ce qu'il peut savoir AVEC étiquettes

```
client : 82.12.34.56
protocole : https
host : datashare.exemple.fr
```

⚠ **Ce que DataShare en fait réellement** : *rien pour l'instant*. Une recherche dans `backend/src/main` ne trouve ni lecture de `X-Forwarded-*` ni `forward-headers-strategy`. Les en-têtes sont envoyés, prêts à servir (journaux, limitation de débit, HTTPS), mais le backend ne s'en sert pas aujourd'hui. De même, le lien de partage est construit à partir de la variable `DATASHARE_DOWNLOAD_BASE_URL`, **pas** à partir de `Host` : c'est la limite documentée dans [docs/BACKLOG.md](#). Le HTTPS est lui aussi prévu « au reverse-proxy », mais pas encore en place (`listen 80` uniquement).

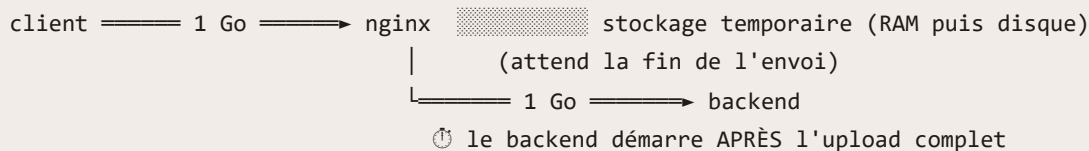
🧠 Pour un enfant de 5 ans

Quand tu envoies une lettre par la standardiste, elle colle des **post-its** dessus : « vient de Léo », « était pour la boutique jouets ». Sinon le grand croirait que c'est la standardiste qui lui écrit. (Pour l'instant, le grand reçoit les post-its mais ne les lit pas encore.)

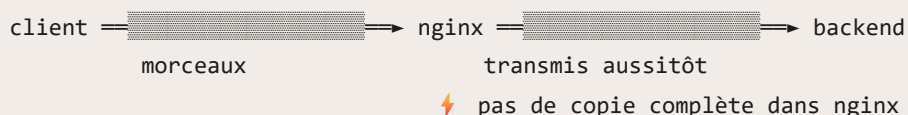
proxy_request_buffering off : streaming de l'upload

Par défaut, nginx **lit tout le corps de la requête** (le fichier) avant d'appeler le backend. Il le garde en mémoire puis dans un fichier temporaire sur disque. Avec `off`, il transmet les morceaux au fur et à mesure.

PAR DÉFAUT : proxy_request_buffering on "je ramasse tout, puis je livre"



DataShare : proxy_request_buffering off "je fais passer au fur et à mesure"



	on (défaut)	off (DataShare)
Stockage dans nginx	tout le fichier	quasi rien
Démarrage du backend	après l'upload complet	dès le premier morceau
Rejet précoce (401, 413...)	tardif	immédiat
Rejouer la requête sur un autre backend	possible	impossible (non pertinent : un seul backend)

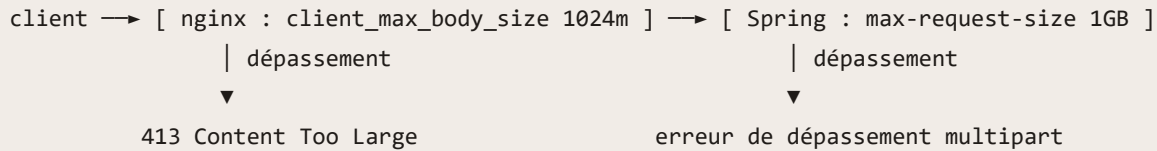
Pour un service dont la limite est 1 Go par fichier, c'est le choix cohérent : pas de 1 Go de fichier temporaire par upload simultané dans le conteneur nginx.

🧠 Pour un enfant de 5 ans

Au lieu de ramasser **toutes** les briques de Lego avant de les donner au grand, la standardiste les lui passe **une par une** dès qu'elle les reçoit. Le grand construit tout de suite, et elle n'a pas besoin d'un énorme coffre.

Les limites : taille et temps

Deux garde-fous encadrent un upload, un par couche. Ils doivent rester cohérents.



Réglage	Fichier	Valeur
<code>client_max_body_size</code>	nginx.conf:8	1024m
<code>spring.servlet.multipart.max-file-size</code>	application.yml:16	1GB
<code>spring.servlet.multipart.max-request-size</code>	application.yml:17	1GB

Sans `client_max_body_size`, nginx refuserait tout corps de plus de **1 Mo** (défaut). La limite porte sur le corps entier de la requête : un fichier de 1 Go pile, avec l’enveloppe multipart, peut être refusé. On annonce donc « jusqu’à environ 1 Go ».

Timeouts : aucun n’est configuré dans le dépôt (ni nginx, ni Spring). Les défauts nginx de 60 s s’appliquent à `client_body_timeout`, `proxy_send_timeout` et `proxy_read_timeout`. Ils mesurent l’inactivité (60 s sans octet), pas la durée totale : un upload lent mais continu passe. Deux risques résiduels : une coupure réseau de plus de 60 s (erreur 408), ou un backend qui met plus de 60 s à répondre après réception (erreur 504). Piste non appliquée : `proxy_read_timeout` et `proxy_send_timeout` à 300 s dans le bloc `/api/`.

Dans DataShare

- Le bloc proxy : [frontend/nginx.conf:19-26](#).
- Limite d’upload nginx : [frontend/nginx.conf:8](#) ; limites Spring : [application.yml:16-17](#).
- Règle regex concurrente justifiant `^~` : [frontend/nginx.conf:36](#).
- Image : nginx `1.29-alpine`, config copiée en `/etc/nginx/conf.d/default.conf` ([frontend/Dockerfile](#)).
- Équivalent en développement : [frontend/proxy.conf.json](#), consommé par `ng serve`.
- Appels du front en chemin relatif (`apiUrl: '/api'`), d’où l’absence de CORS (Cross-Origin Resource Sharing) entre le front et l’API en production : tout passe par la même origine.

À retenir

- `location ^~ /api/` envoie au backend tout ce qui commence par `/api/`, et `^~` empêche la règle regex des assets `.js / .css` de s’en mêler.
- `proxy_pass http://backend:8080;` sans chemin conserve l’URL telle quelle ; `backend` est un nom de service Docker, et seul nginx est exposé.
- Les `proxy_set_header` transmettent l’origine réelle (IP, hôte, protocole) ; le backend ne les exploite pas encore, il faut le dire honnêtement à l’oral.
- `proxy_request_buffering off` fait passer les uploads en streaming : pas de copie de 1 Go dans nginx, et rejet précoce possible.
- La limite de 1 Go est alignée entre nginx (`1024m`) et Spring (`1GB`) ; les timeouts restent aux défauts de 60 s d’inactivité.